

vossie

Developer Series 1

Prototype Architecture & Multi-Agent Development System

Technical implementation blueprint • Decisions 01–02

Purpose. Define how Vossie is distributed as an adaptive prototype and how ChatGPT, Cursor, Cursor-native agents/skills, and VS Code work together without architectural drift.

Status. Decisions 01 and 02 confirmed. Full stack selection remains Decision 03.

September 2026 • Version 2

Decision 01 — Single-deployment adaptive PWA

Vossie uses one URL/QR entry point and one shared application platform. The interface adapts to available screen space and capabilities rather than maintaining separate phone and desktop products. All clients share authenticated backend state, APIs, data and realtime events.

Surface	Experience	Primary use
Phone	Mobile-first shell, bottom navigation, Reels, camera/QR, courier map	Buyer / Seller / Courier
Tablet	Adaptive split views and contextual panels	Marketplace + light Operations
Windows / macOS	Desktop shell, sidebar, multi-column views, maps and tables	Seller Studio / Hub / Incubation / Admin
Fallback browser	Responsive web access if installation is unavailable	Universal access

One URL. One application platform. One backend. Multiple adaptive experiences.

Prototype external dependencies are isolated behind adapters so sandbox payment, simulated institutional identity, simulated GPS and demo notifications can later be replaced without rebuilding Vossie's domain model.

Decision 02 — Multi-agent development architecture

The development model separates architectural coordination, implementation and independent verification. ChatGPT acts as the cross-environment second brain and technical architecture/orchestration layer. Cursor is an implementation environment that contains both the ChatGPT-assisted workflow and Cursor-native agents/skills. VS Code is the engineering control and verification environment.

The agents do not rely on shared conversational memory. Git, repository specifications, ADRs, contracts, tests and documented handoffs form the persistent source of truth.

A. ChatGPT — architecture and orchestration layer

Responsibility	Output
System architecture	Boundaries, domain design, architecture diagrams and technical decisions
Planning	Feature decomposition, tickets, acceptance criteria and dependency order
Contracts	API/data/event expectations and cross-application behavior
Security architecture	Threat-aware requirements, authorization boundaries and review gates
Review support	Reason over diffs, failures and agent handoffs against the Vossie blueprint
Technical memory	Keep Developer Series decisions aligned with repository documentation

B. Cursor — implementation environment

Cursor contains a ChatGPT-assisted developer workflow plus Cursor-native specialized agents and reusable skills. Cursor agents are allowed to implement code within assigned scopes; they do not independently redefine Vossie architecture.

Cursor agent	Primary tasks	Core skills/context
Frontend	Marketplace/Operations UI, adaptive layouts, PWA behavior	UI system, TypeScript, accessibility, responsive rules
Backend + Database	Services, APIs, validation, migrations, queries	API contracts, Postgres, authorization, transactions
Marketplace + Logistics	Listings, Requests, orders, courier, Hub, transfer/custody	Marketplace domain, logistics states, QR/OTP, realtime
Integrations + Realtime	Provider adapters, events, subscriptions, sandbox integrations	Realtime, identity, payments, maps, notifications

C. Cursor skill library

Skills are reusable procedures/context packs rather than autonomous owners. Agents compose the skills required by their ticket.

Shared skill	Purpose
vossie-architecture	Domain boundaries, application families, dependency rules and ADR requirements
vossie-security	Authorization, secrets, input validation, sensitive operations and security gates
vossie-database	Schema conventions, migrations, constraints, RLS and query practices
vossie-api-contracts	Request/response/event contracts and versioning rules
vossie-ui-system	Design tokens, adaptive patterns, accessibility and component conventions
vossie-testing	Required unit/integration/E2E coverage and fixtures
vossie-realtime	Event naming, subscription scope, authorization and synchronization
vossie-pwa	Manifest, install behavior, service-worker/offline rules and fallbacks
vossie-payments	Ledger invariants, provider callbacks, idempotency and protected payment states
vossie-logistics	Courier eligibility, shipment states, manifests, custody, QR/OTP
vossie-git-workflow	Branches, commits, PR requirements and protected main
vossie-definition-of-done	Tests, documentation, security checks and handoff requirements

D. VS Code — verification and engineering control

VS Code agent/skill	Primary responsibility
Security	Authorization/RLS review, OWASP/API risks, secrets and dependency/security findings
QA / Testing	Unit, integration, E2E, regression and failure-path validation
Review / Debug	Inspect Cursor diffs, architecture compliance and trace runtime failures
Git / Release	Branches, PR gates, versioning, CI/CD and release checks
Optional specialist checks	Performance, accessibility, database audit and API-contract drift

E. Engineering workflow

1. Developer selects a feature. **2.** ChatGPT decomposes it into architecture, contracts, risks and implementation tickets. **3.** Cursor-native agents implement scoped tickets using shared skills. **4.** Work is committed to a feature branch and submitted as a PR. **5.** VS Code verification agents inspect security, tests, contracts and runtime behavior. **6.** CI executes automated gates. **7.** Failures return to implementation; approved work merges to protected main.

Cursor builds; VS Code verifies; ChatGPT coordinates architecture; Git and documented specifications preserve truth.

F. Mandatory agent boundaries

Rule	Requirement
Protected main	No agent pushes directly to main.
Scoped work	Agents modify only ticket-authorized areas unless escalation is documented.
Architecture	Agents cannot silently change architecture; significant changes require an ADR.
Database	Migrations require review; destructive changes need explicit approval.
Auth/security	Authentication, authorization and RLS changes require security verification.
Payments	Ledger/payment changes require dedicated invariants, idempotency and failure-path tests.
Tests	Agents may not remove/disable tests merely to make CI pass.
Secrets	No secrets in prompts, source, commits or logs.
Completion	Feature = implementation + tests + docs + security/quality checks + handoff.

G. Standard agent handoff

Every implementation handoff must state: ticket/branch; files changed; migrations; APIs/events changed; environment variables; tests added; test results; security considerations; known limitations; unresolved questions; and the exact verification requested from the next agent.

H. Initial team size

Start with four Cursor build agents — Frontend; Backend/Database; Marketplace/Logistics; Integrations/Realtime — and four VS Code verification roles — Security; QA/Testing; Review/Debug; Git/Release. Split into more specialized agents only when project complexity justifies it.

Repository source-of-truth structure

```
/docs/architecture  /docs/adr    /docs/api    /docs/database  /docs/security
/docs/ui-ux        /agent-context
```

The agent-context area should contain architecture rules, coding standards, security rules, database rules, UI rules and the definition of done so both IDE workflows reference the same engineering constraints.

Next decision

Developer Series 1 — Decision 03: Full technology-stack selection. The next stage will select and validate the frontend, backend, database, auth, realtime, storage, maps/GPS, QR, video/Reels, notifications, payments, testing, observability, hosting and CI/CD layers, then assign ownership and verification responsibility to the agent system defined above.

Decision 03 - Full Stack Architecture

This addendum extends the confirmed Decisions 01-02. It separates the locked core from provider choices that remain intentionally swappable, and adds the formal IDE/workstation tooling required by the multi-agent development model.

A. Locked core stack

Layer	Selected direction	Purpose
Language	TypeScript	Shared frontend/backend types and safer agent-generated code
Web/PWA	Next.js + React	Universal adaptive application across mobile and desktop
UI	Tailwind CSS + Radix/shadcn-derived Vossie system	Fast accessible primitives without surrendering Vossie's visual identity
Backend	TypeScript modular monolith	Clear domains without premature microservices
Database	PostgreSQL	Relational integrity for marketplace, orders, ledger and logistics
Prototype platform	Supabase	Managed Postgres, Storage, Realtime and supporting backend capabilities
Data access	Drizzle + explicit SQL where needed	Typed queries/migrations while retaining Postgres visibility
Validation	Zod	Runtime validation at trust boundaries
Authorization	Vossie RBAC/capabilities + server checks + RLS	Object-level and campus/resource-scoped authorization
Realtime	Supabase Realtime	Orders, Requests, courier/logistics and selected operational updates
Testing	Vitest + Playwright	Unit/integration logic plus cross-browser end-to-end workflows
Repository	GitHub monorepo; pnpm + Turborepo	One source of truth for both IDEs and all agents
CI/CD	GitHub Actions	Lint, typecheck, tests, security gates and builds
Prototype deploy	Vercel + Supabase	Fast preview/staging/demo deployment

B. Identity: Clerk added as a formal candidate

Clerk is added behind a Vossie Identity Provider boundary. It is not layered on top of Supabase Auth by default. The team will select Clerk or Supabase Auth for the prototype identity/session layer after validating institutional federation, authorization mapping, cost, developer experience and migration path.

Prototype choice	Role	Decision status
Clerk	Candidate application auth/session provider	Evaluate before lock
Supabase Auth	Candidate auth tightly integrated with Supabase/Postgres	Evaluate before lock
Institutional OIDC/SSO	Production target when formally approved by Eduvos	External approval/integration pending

Regardless of provider, Vossie owns its profile, role/capability and resource-scope model. Institutional passwords and authenticator secrets are never collected by Vossie.

C. SwiftUI native iOS expansion layer

SwiftUI is added to the architecture as an optional native iOS layer, not as a replacement for the Next.js PWA. The PWA remains the universal prototype and desktop/mobile baseline. SwiftUI becomes justified when validated requirements exceed reliable web/PWA capability - for example richer Apple-specific UX, native camera/scanning, stronger native notification behavior or courier location/background workflows.

Layer	Use now	Use later
Next.js PWA	Primary universal prototype and installable web experience	Retained as web/desktop surface
SwiftUI	No duplicate full app required for initial prototype	Native iOS shell/features when evidence requires them
Android native strategy	PWA first	Choose React Native/Expo or Kotlin/Compose after native requirements

D. Provider/adaptor decisions still open

Area	Prototype direction	Production decision
Authentication	Clerk vs Supabase Auth evaluation	Approved institutional federation where available
Payments	Sandbox PaymentProvider	South African marketplace-capable PSP after due diligence
Video/Reels	Controlled object storage and limits	Dedicated media/video pipeline if scale warrants
Courier location	Simulated/foreground prototype events	Native/background strategy after privacy, battery and OS validation
Email/push	Adapter-based demo/provider candidate	Provider selected by deliverability, cost and compliance
Observability	Structured logs; Sentry candidate	Final monitoring stack after evaluation

E. Developer Workstation & IDE Tooling

Plugins are productivity layers, not architectural dependencies. The repository must remain buildable/testable through documented CLI commands even when an IDE extension is unavailable.

VS Code approved tooling profile

Area	Extension/tool class	Purpose
Git	GitLens	History, blame and commit/branch context
GitHub	GitHub Pull Requests and Issues	PR and issue review inside the editor
Quality	ESLint + Prettier	Static rules and deterministic formatting
UI	Tailwind CSS IntelliSense	Class completion and validation
Database	PostgreSQL database client	Inspect development/staging data and queries
Backend	Supabase CLI/tooling	Local/project backend workflow
API	REST Client or Thunder Client	Exercise and inspect API endpoints
E2E	Playwright Test for VS Code	Run/debug browser workflows
Unit tests	Vitest integration	Run/debug unit and integration tests
Environment	Dev Containers + Docker tooling	Reproducible development environment where useful
Security	Snyk/equivalent + secret scanning	Dependency/code findings and credential leak prevention
Diagnostics	Error Lens	Surface diagnostics inline
Collaboration	Live Share where useful	Pair development and debugging

Repository-managed VS Code configuration

Check in `.vscode/extensions.json`, workspace settings, tasks and debug configurations so a new developer or agent receives the same recommended workstation baseline. Extensions should be reviewed periodically; redundant or abandoned plugins should be removed.

Cursor tooling profile

Cursor remains the primary AI implementation environment. Cursor-native agents and reusable skills are versioned with the project where supported. They consume the same architecture, security, API, database, UI, testing and definition-of-done context used by the ChatGPT orchestration layer and VS Code verification workflow.

Required CLI foundation across both IDEs

Node.js | pnpm | Git | GitHub CLI | Supabase CLI | Vercel CLI | TypeScript | Playwright | lint/test/build scripts

The command-line foundation prevents the development process from depending on a specific plugin and gives CI the same executable workflow developers and agents use locally.

F. Ownership under Decision 02

Stack area	Cursor implementation owner	VS Code verification owner
Next.js / UI / PWA	Frontend Agent	Review/Debug + QA + Accessibility
Postgres / Drizzle / RLS	Backend/Database Agent	Security + Database Audit
Marketplace / Logistics	Marketplace/Logistics Agent	QA + Security + Review
Realtime / provider adapters	Integrations/Realtime Agent	API Contract + Security + QA

Stack area	Cursor implementation owner	VS Code verification owner
SwiftUI expansion	Dedicated native agent when activated	Native QA/Security review
Auth provider	Backend/Integration Agent	Security Agent
CI/CD and releases	Implementation agents supply tests/config	Git/Release Agent owns gates

Decision 03 status

Locked: TypeScript, Next.js/React PWA, PostgreSQL/Supabase foundation, Tailwind-based Vossie design system, Drizzle, Zod, capability/RLS authorization, Supabase Realtime, GitHub monorepo, pnpm/Turborepo, Vitest/Playwright, GitHub Actions and Vercel/Supabase prototype deployment.

Formally added: SwiftUI as the optional native iOS expansion layer; Clerk as an authentication candidate behind the identity-provider boundary; and the VS Code/Cursor workstation tooling specification.

Pending focused research: final auth provider, production PSP, production video pipeline, background courier-location strategy, notification provider and final observability provider.